# SCULPTOR Has Your Back(up Path): Carving Interdomain Routes to Services

Paper #365, 12 pages body, 20 pages total


## Abstract

Cloud and content providers (Service Providers) serve diverse Internet applications — enterprise services, gaming, video, real-time AI inference — with goals that pull in different directions: low latency for some flows, high availability for others, low cost for bulk transfers, and graceful behavior during failure. To meet these goals Service Providers either overprovision, react too late, or propose narrow solutions. We take a different approach with SCULPTOR, a framework that proactively optimizes BGP configurations to satisfy diverse ingress routing objectives-including latency, cost, and capacity constraints. SCULPTOR models Internet routing to approximately solve a large integer optimization problem at scale using gradient descent. We prototype SCULPTOR on the PEERING testbed at 32 sites of a global public cloud and demonstrate its efficacy on a variety of objectives. On the live Internet, SCULPTOR serves users within 1.9 ms of an optimal configuration requiring 19× more prefixes, closing 65% of the gap left by the best prior approach; in emulation it overloads 40% less traffic during site failures and carries 1.6× more bulk traffic than other approaches.


## 1 Introduction

Cloud/content providers (hereafter Service Providers) enable diverse Internet applications used daily by billions of users. Traditionally, Service Providers use DNS and static BGP advertisements to define a single path from a user to a service, leaving the specific path up to the Internet to determine.

Increasingly, however, these services have diverse requirements that can be challenging to meet with a single path that Service Providers have little control over. For example, enterprise services have tight reliability requirements [1, 2], and new applications such as virtual reality require ≤ 10 ms round trip latency [3] and ≤ 3 ms jitter [4]. Complicating matters, Service Providers must meet these requirements subject to changing conditions such as peering link/site failures [5, 6] and DDoS attacks [7–9]. Critically, such changes can cause *overload* if the Service Provider cannot handle new traffic volumes induced by the change which can lead to degraded service for users.

Current solutions are either (a) too costly, (b) too reactive, or (c) too specific. For example, Service Providers take a *costly* approach by proactively overprovisioning resources [10–12], but we demonstrate that bursty traffic patterns can require overprovisioning rates as high as 70% (§2.3). TIPSY *reactively* responds to changing loads to retain reliability [6], but reactivity could lead to short-term degraded performance. AnyOpt/PAINTER proactively set up routes to optimize *specific* objectives such as steady-state latency [1, 13], but it is challenging to extend those approaches to other objectives (§5.4).

We present Service Providers with a flexible framework, SCULPTOR (**S**couring **C**onfigurations for **U**tilization-**L**oss-and **P**erformance-aware **T**raffic **O**ptimization & **R**outing), which accepts as input general objectives and outputs BGP advertisements and traffic allocations that help achieve those desired objectives. SCULPTOR is the first system that finds good *ingress interdomain* routes for Service Provider objectives such as maximum link utilization, cost, and latency (existing systems work for intradomain traffic, *e.g.,* [14–17]). SCULPTOR optimizes for any objective for which, given fixed routes, an allocation of traffic to routes is efficiently solvable. SCULPTOR computes BGP advertisements proactively without affecting live traffic, only placing live traffic on new routes after convergence, mitigating risk (§4).

SCULPTOR makes finding good interdomain routes tractable with two key design decisions. First, since interdomain paths are hard to predict and slow to measure, SCULPTOR maintains a probabilistic model of how users route and refines it with occasional BGP advertisements (§3.4). SCULPTOR can thus estimate objectives for arbitrary configurations without announcing them, so it assesses millions of candidates rather than the thousands that the greedy and random searches in prior work do [1, 13]. Second, SCULPTOR's estimates may be noisy since routing is hard to predict, which SCULPTOR solves via gradient descent (§3.5.2). Gradients allow SCULPTOR to

accumulate evidence about how useful each advertisement is across iterations, aiding noise rejection.

We prototype and evaluate our framework at Internet scale using the PEERING testbed [18] (§4), which is deployed at 32 Vultr cloud locations [19]. Vultr is a global public cloud that allows our prototype to issue BGP advertisements via more than 10,000 peerings. We make an anonymized implementation of SCULPTOR available at https://anonymous.4open.science/r/sculptor_stripped-65F6 and will release all code.

We demonstrate SCULPTOR's utility across diverse objectives with thorough comparisons to prior approaches. Evaluations include both an on-Internet deployment (§4.2) and more thorough emulations (§4.3). On the Internet, measured from RIPE Atlas probes, SCULPTOR cuts latency relative to the best prior approach by 3.6 ms in steady state, 6.3 ms during a link failure, and 12 ms during a site failure, failures under which the best prior approach overloads 75.8% and 95.0% of traffic respectively (§5.2).

In emulations seeded by measurements from tens of thousands of networks (§5.3), SCULPTOR is universally the best deployable solution. It achieves near-optimal max-link-utilization, and achieves steady-state latency within 1.3 ms of optimal. Compared to the best prior approaches: SCULPTOR achieves 19% lower latency during link failures, overloads 40% less traffic, routes 22% less traffic beyond 10 ms of its optimal latency, carries 1.6× the bulk traffic before high-priority traffic congests, and lowers traffic site cost by 6%.

After decades of intradomain traffic engineering using programmable networking primitives such as virtual output queueing for differentiated service [20] (and others, §7), SCULPTOR brings such control closer to realization in the interdomain setting with a unilaterally deployable framework. Service Providers can use flexible frameworks such as SCULPTOR to help bring us the resilient, performant service that our diverse applications increasingly need.

# 2 Motivation

## 2.1 Variable, Evolving Goals

Evolving Internet use cases are pushing Service Providers to meet diverse requirements. For example, Service Providers increasingly host mission-critical services such as enterprise solutions [1], which are predicted to be a $60B industry by 2027 [21]. These enterprise solutions require high reliability, whereas low latency is a more important metric for gaming [3], which generates more revenue than the music and movie industries combined [22]. Service Providers are also expanding the set of services they provide—for example, CDNs that traditionally hosted static content are pivoting to offering services like compute [23–25].

Moreover, as use cases evolve, Service Providers increasingly need to meet performance requirements for *ingress* traffic since that traffic includes, for example, player movements in real-time games, voice and video in enterprise conferences, and video/image uploads for AI processing in the cloud [26, 27]. This reality is a departure from traditional CDN traffic patterns in which ingress traffic was primarily small requests and TCP acknowledgments.

Despite efforts to meet variable service requirements [1, 6, 12, 28–31], doing so is still challenging due to issues *outside* Service Provider control.

First, Service Providers lack control over which ingress path traffic takes because the source and upstream networks determine the route. BGP computes paths in a distributed fashion where each AS selects the single best path to the destination and propagates it; Service Providers cannot force a source network to select a different path if the source prefers an alternative based on its own local preference. Service Providers cannot even unilaterally control the part of the path closest to them—upgrading peering capacity requires coordination among multiple parties [32].

Second, dynamic factors outside Service Provider control make satisfying requirements even harder. Peering disputes can lead to congestion on interdomain links and inflated paths [5, 33, 34], and DDoS attacks still bring down sites/services [9, 35–37], despite considerable effort in mitigating attacks [7, 8]. Moreover, recent work [17, 38, 39] and blog posts [40–45] show that user traffic demands are highly variable due to flash crowds and path changes and so can be hard to plan for.

## 2.2 Standard Approaches

To meet evolving goals, today most Service Providers either use anycast prefix advertisements to provide low latency and high availability at the expense of some control [46–49], or unicast prefix advertisements to direct users to specific sites [32, 50–52]. Anycast, where Service Providers advertise a single prefix to all peers/providers at all/many sites, offers some natural availability following failures, since BGP automatically reroutes most traffic to avoid the failure after tens of seconds [48]. Prior work shows that this availability comes with higher latency in some cases [47, 53]. Unicast gives clients lower latency than anycast by advertising a unique prefix at each site [50, 52], but can suffer from reliability concerns, taking as much as an hour to shift traffic away from bad routes due to stale DNS records [1].

## 2.3 New Systems & Their Limitations

In response to some of the limitations of anycast and unicast, others have proposed specialized solutions. However, these solutions have three key limitations.

**Too Specific:** Some have noted that, by intelligently advertising different prefixes to subsets of peers/providers to offer users more paths, Service Providers can achieve better performance [1, 5, 13, 46, 49]. However, these solutions

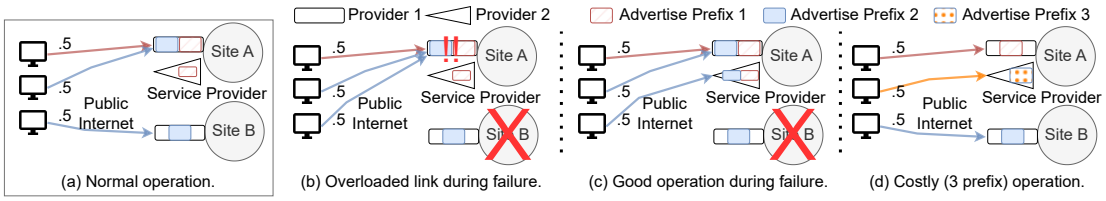



Figure 1: In normal operation traffic is split between two sites by directing half the traffic to each prefix using DNS (a). Even though there is enough global capacity to serve all traffic when site B fails, there is no way to split traffic across multiple providers given available paths, leading to overload (b). A proactive solution is to advertise prefix 2 to an additional provider at site A *a priori*, allowing traffic splitting across the two links (c). A simpler but prohibitively expensive solution is to proactively advertise one prefix per peering (d).

rely on objective-specific solution methods that do not easily generalize to multi-constraint optimization problems (§5.4).

Figure 1 shows how this specific focus on a single objective could lead to reliability issues, *e.g.,* during a site failure. In normal operation, user traffic is split evenly across two prefixes and achieves low latency (Fig. 1a). However when site B fails, BGP chooses the route through Provider 1 for all traffic, causing link overutilization and subsequent poor performance for all users (Fig. 1b). In Section 5 we show that this overload happens in practice for state-of-the-art systems that provide low (steady-state) latency from users to Service Provider networks such as `AnyOpt` [13] and `PAINTER` [1].

**Too Reactive:** Systems like `TIPSY` [6] enhance reliability by withdrawing BGP announcements to drain overloaded links. However, changing BGP is known to lead to outages [12, 54–56], and can take time to converge. We should modify announcements for live traffic only as a last resort.

**Too Costly:** Service Providers often overprovision to handle transient peak loads [10–12], but this approach can incur high costs as Figure 2 demonstrates using link utilization data (per 5 minutes) from OVH cloud [57].

To generate Figure 2 we first split the dataset into successive, non-overlapping 120-day planning periods and compute the $95^{th}$ percentile ingress link utilization ("near-peak load") for each link/period. We then assign each link's capacity for the next period as the near-peak load in the current period multiplied by an overprovisioning factor (X-axis variable). We then compute the average link utilization in the next period and the total number of links on which we see ≥ 100% utilization in at least one 5-minute interval.

Figure 2 plots the median utilization across links and periods and the number of congestive events as we vary the overprovisioning factor. Figure 2 shows that overprovisioning to accommodate peak loads introduces a tradeoff between inefficient utilization and overloading. Low overprovisioning factors between 10% and 30% lead to more efficient utilization (60%-70%) but lead to thousands of congestive events. High overprovisioning factors lead to far less overloading, but only 50% utilization. These events span 41 OVH sites and 1,650 links, indicating a widespread challenge. Results were consistent with larger planning periods, while smaller periods increased congestion.

**An important note on cost.**: We model deployment cost as

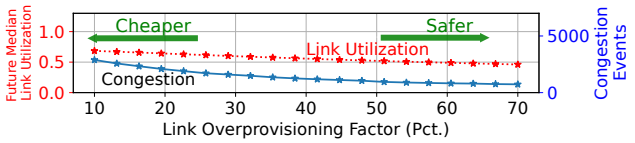



Figure 2: Planning for peak loads to avoid overloading requires inefficient overprovisioning.

correlated with the peering capacity over all links/sites, even though the actual relationship may be complex and additional peering capacity itself is not prohibitively expensive. Upgrading capacity has fixed costs (*e.g.,* fiber, router backplane bandwidth, line card upgrades, CDN servers near peering routers) and variable costs (*e.g.,* power, transit), both of which scale with demand [11, 17, 58, 59]. Hence the high "Link Overprovisioning Factor" to ensure low congestion should be extrapolated out to the general "cost" of upgrading Service Provider serving infrastructure, rather than just a peering link.

**Efficient, Proactive Planning is Possible:** Despite these limitations, Figure 1c shows that by advertising prefix 2 to provider 2 at site A a priori, the Service Provider can split traffic between the two links during failure, avoiding overloading without (a) reacting to the failure and (b) upgrading deployment capacity.

## 2.4 Key Challenges

Since a Service Provider has lots of global capacity, dynamically placing traffic on paths to satisfy performance objectives subject to capacity constraints would therefore be simple if all the paths to the Service Provider were always available to all users as in Figure 1d. However, making paths available uses IPv4 prefixes, which are expensive and contribute to the global routing table's memory footprint. The solution in Figure 1d uses 50% more prefixes than the solution in Figure 1c and, at $20k per prefix for 10,000 peerings would cost $200M [60]. Prior work noted the same but found that advertising around 50 prefixes was acceptable [1, 13], and most Service Providers advertise fewer than 50 today according to RIPE RIS BGP data [61]. IPv6 is not a good alternative since it is not supported by 60% of users according to APNIC [62] and 50% according to Google [63], so Service Providers would require dual-stack solutions.

Since we cannot expose all the paths by advertising a

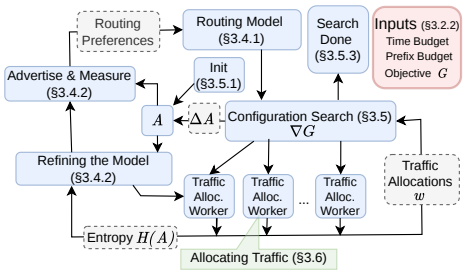



Figure 3: SCULPTOR advertisement computation overview.

unique prefix to each connected network, a Service Provider must therefore choose which subset of paths to expose, out of more than $2^{10,000}$ possibilities at 10,000 peerings [64]. Section 3 describes why searching this space of possibilities is hard and how SCULPTOR solves this challenge.

# 3 Methodology

## 3.1 Overview

SCULPTOR takes as input a Service Provider's objective, its prefix budget, and a total training time (§3.2.2). It yields BGP advertisements that give users good interdomain paths relative to that objective, and an allocation of traffic to those paths. SCULPTOR computes advertisements proactively, only placing live traffic on paths after termination (§4).

To search for good advertisements and traffic allocations, SCULPTOR solves two key challenges. First, SCULPTOR cannot easily compute the objective. The only way to know exactly how users will route to a configuration is to advertise it, which can be done at most a few times per hour and so cannot be done for the millions of configurations that an optimization considers. SCULPTOR instead maintains a probabilistic (*i.e.,* noisy) model of how users route, uses it to estimate the objective, and refines it with occasional measurements (§3.4).

Second, the configuration space is large and discrete. SCULPTOR searches it using gradient descent over a relaxed, real-valued configuration which helps smooth the convergence process under a noisy objective (§3.5). Each gradient entry is computed by solving a small traffic-allocation problem (§3.6). Figure 3 shows how these pieces fit together, and in Section 5.4 we demonstrate that each piece is important for SCULPTOR's success. We also outline a full iteration of SCULPTOR in Section A.3.

## 3.2 Problem Background

### 3.2.1 Setting and Routing Model

Service Providers offer their services from tens to hundreds of geo-distributed sites [65]. The sites for a particular service can serve any user, but users benefit from reaching a low-latency site for performance. Sites consist of sets of servers that have an aggregate capacity. Service Providers also connect to other networks at sites via dedicated links or shared IXP fabrics that we call peering links. Each such link also has a capacity. When utilization of a site or link nears/exceeds the capacity, performance suffers, so Service Providers strive to avoid high utilization [5, 17]. Resources can also fail due to, for example, physical failure and misconfiguration.

Users route to the deployment through the public Internet to a prefix over one of the peering links via which that prefix is advertised. The path to a prefix (therefore peering link) is chosen via BGP. We model interdomain paths from users to the Service Provider as non-overlapping except at the peering link through which each path ingresses to the deployment. We identify a path by its ingress link rather than its neighbor AS, because one AS may peer at many sites.

We assume that the peering link is the bottleneck of each path, for two reasons. First, this link increasingly represents the most important part of the path for Service Providers due to Internet flattening [66]. Second, other parts of the path are less interesting for traffic engineering—last-mile bottlenecks are common to all paths for a user, and existing systems perform traffic engineering in intermediate networks. Handling unexpected bottlenecks (*i.e.,* in a peering network) can also be viewed as a path failure, which we evaluate in Section 5.3.2, *demonstrating that SCULPTOR handles violations of this assumption*.

We assume the Service Provider has some way to direct each UG's traffic among its prefixes, such as DNS [46, 50–52], multipath transport [67–69], or control points near users [1, 70]. These options differ in how deployable and how precise they are as PAINTER discusses [1]. SCULPTOR makes no particular assumption about which is used.

### 3.2.2 Inputs

The high-level input is the Service Provider's *goal* which they express via an objective function, $G$. The Service Provider hopes to accomplish this goal via a *prefix budget* which is generally much less than the number of total ingresses, and a maximum *training time* or, equivalently, a maximum number of BGP measurements, since SCULPTOR is bottlenecked by BGP measurement time (§3.4).

Oftentimes, objectives require ancillary information about the deployment or the Internet. This information depends on the objective, but we discuss common cases that we use in our evaluated objectives (§5.1.2). Useful metrics (*e.g.,* latency, cost) are readily attainable through methods developed in prior work, and obtaining others is an orthogonal problem.

We aggregate users into user groups (UG*s*), for example /24s, metro-AS pairs, or ASes. The granularity is chosen to be one at which all users in a UG route to the Service Provider the same way, so that a UG has one route per prefix (§3.3). We use /24s (§4). UG*s* generate steady-state traffic volumes, $v(\text{UG})$, and the Service Provider provisions capacity at links/sites to accommodate this load. A system run by the

| | |
|---|---|
| UG | Users that route alike (*e.g.,* a /24) |
| $v(\texttt{UG})$ | Traffic volume of UG |
| $c(l)$ | Capacity of ingress link $l$ |
| $\mathcal{L}(\texttt{UG}, l)$ | Latency for UG via link $l$ |
| $A(p, l)$ | 1 if prefix $p$ is advertised over link $l$ |
| $w(p, \texttt{UG})$ | Traffic from UG directed to prefix $p$ |
| $R(A)(p, \texttt{UG})$ | Ingress link UG uses for $p$ ($\emptyset$ if none) |
| $G(R(A), w)$ | Objective value under $A$ and $w$ |

Table 1: Notation used throughout Section 3.

Service Provider measures latency, $\mathcal{L}$, and traffic volumes, $v$, from UGs to Service Provider peering links $l$ with known capacities $c(l)$, which are also known inputs in prior work [1, 13, 52, 71–73]. Part of these latency measurements includes measuring which peering links are *reachable* (in a policy-compliant routing sense) from each UG.

## 3.3 The Key Goal

Finding advertisement configurations that admit good assignments of user traffic to paths is a multivariate optimization problem over both configurations and traffic assignments. We tabulate variables and expressions in Table 1.

We represent an advertisement configuration $A$ as a binary vector. Each entry indicates whether we advertise/do not advertise a particular prefix to a particular peer/provider at a particular site (similar to prior work [1, 13]). Implementing this configuration (*i.e.,* advertising prefixes via BGP sessions) results in routes from UGs to the Service Provider. A UG has one route per prefix, but a UG may have their traffic fractionally assigned to different prefixes. The assignment of traffic to resulting routes from this configuration is given by the non-negative real-valued vector $w$, whose entries specify traffic allocation for each UG along each route (equivalently, prefix).

Announcing a configuration results in some set of routes—though knowing which routes a priori is challenging—and those routes determine which ingress link a UG uses for each prefix. We formalize this idea as a routing function $R$ mapping a configuration to an assignment of $\langle \texttt{prefix}, \texttt{UG} \rangle$ pairs to ingress links, so that $R(A)(p, \texttt{UG}) = l$ means users in UG reach prefix $p$ over link $l$ under configuration $A$. Because a configuration may leave a UG with no route to a prefix, we define an indicator $e(R(A)(p, \texttt{UG}))$ that is 1 when a route exists and 0 otherwise.

Now suppose that the overall metric we want to minimize is $G$ which is a function of both configurations and traffic assignments. Examples include traffic cost, average latency, maximum latency, and their combinations (§5.1.2). The joint minimization over configurations and traffic assignments can then be expressed as the following.

$$\begin{aligned} \min_{A,w} \quad & G(R(A), w) \\ \text{s.t.} \quad & w(p, \texttt{UG}) \geq 0; \quad A(p, l) \in \{0, 1\} \quad \forall \texttt{UG}, p, l \\ & \sum_p w(p, \texttt{UG}) e(R(A)(p, \texttt{UG})) = v(\texttt{UG}) \quad \forall \texttt{UG} \end{aligned} \tag{1}$$

The first constraint requires that traffic assignments be non-negative and that configurations are binary. The second constraint requires that all user traffic $v(\texttt{UG})$ is assigned, and none is assigned to nonexistent paths. Capacity constraints depend on the choice of $G$.

Solving Equation (1) is challenging because we need to compute $G(R(A), w)$ but measuring $R(A)$ exactly requires advertising prefixes in the Internet, which can only be done infrequently to avoid route-flap-dampening and allow convergence. The optimization problem requires evaluating $G(R(A), w)$ for millions of different $A$, which could take a hundred years at a rate of advertising one configuration every 20 minutes.

Moreover, even if $R$ were known, Equation (1) is not a tractable optimization problem since (1) $G$ depends on $A$ only through $R$, the Internet's routing rather than an algebraic function of $A$. Therefore, we cannot express how $A$ determines each UG's ingress as constraints over $A$, and (2) even were it expressible, the resulting MIP is too large for off-the-shelf solvers (§5.4). SCULPTOR addresses these challenges by decomposing the problem as follows.

$$\min_A \quad \mathrm{E}_R \Big[ \min_w G(R(A), w) \Big] \quad \text{s.t. constraints of Equation (1)} \tag{2}$$

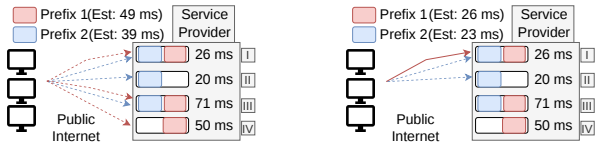
In the remainder of this section we describe each component in Equation (2). The expectation over $R$ represents what we know about routing and how we learn more (§3.4). The outer minimization over $A$ is how we move through configuration space (§3.5.2). The inner minimization over $w$ is how we allocate traffic once routes are fixed (§3.6), and it is where a Service Provider's specific objective, $G$, plugs in.

## 3.4 Modeling & Measuring Routes

In Equation (2) we compute expectations over a random process that represents the routing, $R$, since we cannot exhaustively measure $R$. Specifically, SCULPTOR models UG paths probabilistically (§3.4.1) and improves its model over time through relatively few measurements (§3.4.2).

### 3.4.1 Routing Model

A priori, for a given UG towards a given prefix, SCULPTOR treats all reachable ingress link options to which that prefix is advertised as equally likely. One could incorporate prior information about ingress preferences (from economic/routing




(a) (b)

Figure 4: SCULPTOR initially estimates latency from this UG to both the red and blue prefixes with the average over possible ingress latencies (4a). A priori, no prefix is advertised. SCULPTOR then advertises the red prefix and learns that ingress I is preferred over III and IV (4b). SCULPTOR uses this information to refine its latency estimate towards the blue prefix (ingress III is not a preferred option) without advertising the blue prefix, saving time.

models, for example), but such complexity is unnecessary for finding good solutions.

We instead learn preferences over time via BGP measurements and observe which ingress the UG takes (§3.4.2). Upon learning that one ingress is preferred over the others receiving that prefix advertisement, we exclude that less-preferred ingress as an option for that UG in all future calculations for all prefixes for which both ingresses are an option. Prior work used a similar routing model [1, 13, 74] but did not extend it to deal with general objectives nor did it treat routing probabilistically.

As we learn which paths UGs prefer the most, our estimate of $G$ on unmeasured configurations concentrates on the true value, allowing us to more efficiently carry out Equation (2). An example of this learning process is shown in Figure 4—SCULPTOR refines a latency estimate towards an unmeasured prefix (blue) using measurements towards other prefixes (red). Even though SCULPTOR has 50% confidence in which path the user takes (*i.e.,* in $R(A)$), SCULPTOR knows that the user's latency towards the blue prefix will be about 23 ms (*i.e.,* either 26 ms or 20 ms) and so SCULPTOR obtains a better estimate of the objective.

### 3.4.2 Refining the Model

Probabilistic estimates of $G$ could be inaccurate until we learn UG preferences. To refine its model, SCULPTOR periodically measures $R(A)$ on its current configuration (§3.5) by advertising prefixes and measuring UG preferences by observing which routes users take. To use its BGP measurement budget wisely, SCULPTOR makes two design decisions:

**What to measure:** SCULPTOR measures the *current* configuration (as opposed to random or slightly different ones) since it uses a descent-based optimization (§3.5). This type of optimization compares the current advertisement to many adjacent/different ones, so knowing the current value is much more important than knowing adjacent ones.

**When to measure:** SCULPTOR measures either when (a) its uncertainty (according to Monte Carlo simulations, §3.6) exceeds a certain threshold or (b) SCULPTOR has not executed a measurement in many learning iterations. There are many possible choices of "uncertainty", we choose entropy [75].

## 3.5 Searching Over Configurations

This section specifies the outer optimization in Equation (2) over advertisement configurations, $A$.

### 3.5.1 Initializing $A$

We initialize our configuration to be anycast on one prefix and unicast on remaining prefixes (*i.e.,* one prefix per site), since both are well-understood configurations with complementary strengths (§2.2). If there are not enough prefixes for all sites, SCULPTOR chooses a random subset of sites. Remaining prefixes in the budget (if they exist) are advertised to random subsets of ingresses.

### 3.5.2 Gradient Descent

For the reasons discussed in Section 3.3 we cannot find $A$ simply using an off-the-shelf solver. A reasonable choice to solve such problems are descent-based algorithms—ones that compare adjacent options to the current best guess and pick an adjacent option as the next configuration. However, an *additional* challenge in our setting is that we only have noisy estimates of the objective, so placing too much trust in objective estimates of adjacent configurations can lead to poor convergence.

SCULPTOR instead uses *gradient* descent since gradient descent slowly incorporates and remembers information about how useful each advertisement is rather than overcommiting, which is what greedy and random searches used by prior systems do [1, 13].

To apply gradient descent to our problem, we create a real-valued auxiliary variable to $A$, $\tilde{A}$, with entries between 0 and 1 representing "confidence" values that an advertisement to the corresponding ingress will be beneficial. We threshold $\tilde{A}$ at 0.5 to translate real-valued confidences to whether or not we should advertise a prefix at an ingress (*i.e.,* thresholded $\tilde{A} = A$). For coordinate $(p, l)$, SCULPTOR evaluates the inner objective (§3.6) with $A(p,l)$ set to 1 and then to 0, holding the rest of the (thresholded) configuration fixed. The difference between these two values, scaled by the derivative of a sigmoid at $\tilde{A}$, is the gradient entry (similar to prior work [76]). Since there are too many gradients to compute, we subsample gradient entries and track the largest ones (as in prior work [77]).

We implement SCULPTOR with Nesterov's Accelerated Gradient Descent (§3.5.2) in Python [78]. We use RMSProp with an initial learning rate of 1.0.

**Scaling:** Gradient descent scales with the product of the number of ingresses and number of prefixes but individual entry computation is independent and therefore parallelizable.

**Convergence:** Gradient descent converges to a local minimum for bounded objectives [79]. Our evaluations show that

SCULPTOR finds good solutions over a wide range of topologies (§5.3). Adding richer advertisement capabilities (*e.g.,* BGP community tagging) may lead to better solutions, which is an area for future work.

#### 3.5.3 Termination

SCULPTOR stops gradient descent either when the BGP measurement budget has been exhausted or the average change in objective over iterations falls below a small threshold.

### 3.6 Allocating Traffic

This section specifies the inner optimization over traffic allocations in Equation (2).

Given our routing model (§3.4.1) and current advertisement (§3.5), we Monte Carlo sample routing functions $R$. Each sample independently draws, for every UG and prefix, one ingress uniformly from the ingresses the model still considers possible for that pair (§3.4.1). For each sample we then solve the traffic allocation exactly with a general-purpose solver, and the expectation in Equation (2) is the average of those objectives. In practice we find that a few Monte Carlo samples per gradient entry is sufficient.

Given a sampled routing realization, solving for the optimal traffic allocation is then a problem in intradomain traffic engineering with fixed, known routes. We solve traffic allocations (§3.6) with SciPy HIGHS [80].

**Scaling:** This component scales with the number of Monte Carlo samples and the scaling behavior of each traffic allocation, which depends on the objective, $G$.

**Convergence:** depends on the objective, $G$. SCULPTOR accepts any $G$ for which, given fixed routes, the allocation problem is efficiently solvable.

## 4 Implementation

We prototype SCULPTOR on the PEERING testbed [18], which is available at 32 Vultr cloud sites. We describe how we built SCULPTOR on the real Internet and how we *emulate* a Service Provider for more thorough evaluation.

In practice, Service Providers can use SCULPTOR *safely* by only shifting production traffic onto prefixes after SCULPTOR's announcement exploration has converged and by integrating SCULPTOR into automated route configuration frameworks. More details about deployment details are in Appendix C.

### 4.1 What We Measure & What We Emulate

SCULPTOR's evaluation combines real Internet measurement with emulation, with the distinguishing line determined by the capabilities of public research platforms. Our evaluation measures routes (using PEERING [18]) and latencies (using RIPE Atlas [81]). Traffic volumes and link capacities are emulated because they are not publicly accessible. Prior interdomain work that measures congestion directly was conducted by Service Providers themselves [5, 6, 32, 59, 82–85], while academic work has used methodology like ours [53, 86, 87]. Drawing this line is useful since it keeps our results reproducible and extensible by other researchers on platforms they can access.

### 4.2 On-Internet Deployments

We assess how SCULPTOR performs on the Internet using RIPE Atlas probes [81], which represent a subset of all clients. RIPE Atlas allows us to measure paths (and thus ingress links) to prefixes we announce from PEERING, which SCULPTOR needs to refine its model (§3). We limit the scale of our deployment to 10 sites to avoid reaching RIPE Atlas daily probing limits (15k traceroutes/day). These 10 sites include major hubs across North America, Europe, and South America. Choosing RIPE Atlas probes to maximize network coverage and geographic diversity, we select probes from 972 networks in 38 countries which have paths to 484 unique ingresses. Each probe has paths via approximately 60 ingresses. We use 12 prefixes and change advertisements at most once per hour to avoid route-flap-dampening.

We measure which ingress link a probe took to a prefix using traceroutes. Specifically, we perform IP to AS mappings on traceroutes and identify the previous AS in the path to Vultr. In cases where we cannot infer the ingress link from a traceroute (*e.g.,* unresponsive hops), we use the closest latency from the traceroute to the clients' (known) possible ingresses. For example, if an uninformative traceroute's latency was 40 ms to Vultr's Atlanta site and a client was known to have a 40 ms path through AS1299 at that site, we say the ingress link was AS1299 at Atlanta.

We set all RIPE Atlas probe traffic volumes as equal. We then assign link capacities as a 30% overprovisioned percentage of anycast catchment, similar to prior work [87].

### 4.3 Emulated Deployments

To more thoroughly evaluate SCULPTOR, we emulate hundreds of deployments using measurements from tens of thousands of networks across thousands of random routing scenarios and volume distributions. These emulations are seeded with real latency measurements from targets to Vultr.

**Selecting Targets and Measuring Latencies.** We measure actual latency from IP addresses to our PEERING prototype as in prior work [1], selecting targets according to assumptions about Vultr cloud's client base.

We tabulate a list of 5M IPv4 targets via ping-probing each /24. We then measure latency from all clients to each peer individually by advertising a prefix solely to that peer using Vultr's BGP action communities and pinging clients

from Vultr. Responsive clients are noted as having a policy-compliant path through that ingress.

We limit our focus to clients who had a route through at least one of Vultr's direct peers (we exclude route server peers [88]) since Vultr likely peers with networks with which it exchanges a significant amount of traffic [32]. We found 700k /24s with routes through 1086 of Vultr's direct ingresses. We removed clients whose lowest latency to Vultr was 1 ms or less, as these were assumed to be addresses related to infrastructure, leaving us with measurements from 666k /24s to 825 Vultr ingresses.

**Emulating Traffic Distributions.** We emulate traffic distributions in two ways: First, we randomly choose the total traffic volume of a site as a number between 1 and 10 and then divide volume up randomly among clients that cast routes to that site. Client volumes in a site are set to be within one order of magnitude of each other. We avoid Zipf-like distributions as they lead to trivial solutions where a small fraction of sites dominate importance.

To demonstrate SCULPTOR works using realistic traffic volumes, we also use APNIC AS populations [89], which were shown to be representative of Service Provider traffic volumes [90]. We assign each AS its aggregate population and divide it evenly among its clients. Our measurement targets span 91% of APNIC volume. We then assign link capacities as a 10% overprovisioned percentage of anycast catchment.

**Emulating User Routes.** We emulate UG routes through ingresses according to our preference model (§3.4.1). Prior work found that the ingress preference model held in 90% of the cases they studied [13]. We model UG's preferences among ingresses as completely uniformly random—we found that lower entropy preference distributions (*e.g.,* preferred provider) to be much easier to optimize for since SCULPTOR could optimize a small fraction of routes.

**Random deployment subsampling.** We randomly subsample sites and targets to create diversity across emulations. The distribution of the number of /24s per peer is Pareto-like, so we consider random subsets of /24s through each ingress in a way that balances the number of unique /24s per ingress. Over *all* scenarios, we consider paths from clients in 52k prefixes to 873 ingresses. We use between 10 prefixes for 3 site deployments and 42 prefixes for 32 site deployments, but SCULPTOR's benefits are not specific to a prefix budget (§5.4).

# 5 Evaluation

We evaluate SCULPTOR on the Internet (§5.2), in emulated deployments across all objectives (§5.3), and in micro-evaluations (§5.4). On the Internet, SCULPTOR closes 65% of the 5 ms latency gap to One-per-Peering left by the best prior solution and overloads no traffic when a link or site fails. In emulation, SCULPTOR outperforms all feasible baselines on all metrics (Table 2). Our ablation study shows that these gains are a combination of several methodological contributions, especially probabilistic objective estimation.

Service Providers serve trillions of requests per day, so improving a few percent of traffic by tens of milliseconds represents a significant improvement. Service Providers recently emphasized that small percentage gains are important [91, 92]. Hence, SCULPTOR's gains here reflect Service Providers enhancing their ability to offer services in a meaningful way.

## 5.1 Evaluation Setting

### 5.1.1 Baseline Solutions

Each baseline solution returns a BGP configuration and then solves a generic objective to allocate traffic to routes (§5.1.2). SCULPTOR is the only method to *use* that objective to learn how to best configure routes.

**anycast**: A single prefix announcement to all peers/providers at all sites, which is a common configuration used by Service Providers today [46, 49, 93–97].

**unicast**: anycast on the first prefix, and then a single prefix announcement to all peers/providers at each site (one per site). We add the anycast prefix to create a much more competitive baseline (anycast gives good latency [47]), and since it is similar to configurations that are used in today's deployments [46, 50–52].

**AnyOpt & PAINTER**: Two proposed strategies for reducing steady-state latency compared to anycast [1, 13]. AnyOpt was one of the first works to systematically optimize ingress traffic, while PAINTER built on that effort. (We do not compute the solution for AnyOpt for our evaluations on the Internet since AnyOpt performs poorly in emulations and since AnyOpt takes a long time to compute.)

**One-per-Peering**: A unique prefix advertisement to each peer/provider, so many possible paths are always available from users. This solution serves as our performance upper-bound, even though it is prohibitively expensive. Section 5.4 computes a different optimal on small problem sizes.

### 5.1.2 Objectives Evaluated

SCULPTOR accepts any objective for which, given fixed routes, allocations of traffic to routes are efficiently computable (§3.6). While SCULPTOR likely accommodates other objectives, it provides practical utility at the expense of a formal characterization of feasible functions, similar to prior gradient descent work. Objectives are described here on a high-level and formally presented in Section A.2. Some objectives have multiple terms that trade-off, for example, lower steady-state latency for less congestion during site failures via constant multipliers. Service Providers set these constants according to deployment goals.

In solving for traffic allocations during failure scenarios, links may be overloaded. We say all traffic arriving on a congested link is congested. Table 2 separately notes the

fraction of traffic that lands on congested links and does not include it in average latency computations. For example, an average latency of 30 ms with 5% congestion means that the 95% of traffic that landed on uncongested links had an average latency of 30 ms.

**`Latency + MLU`.** This objective minimizes average steady-state latency plus a constant × max-link-utilization (MLU). We set that constant equal to 0.1. We also set hard constraints that link utilization (computed from the sum of all `UG` traffic arriving on each link) does not exceed 1.0.

**`Dynamic Traffic Failover`.** This objective optimizes `Latency + MLU` during normal operation and single-link/site failures. The objective is thus the sum of `Latency + MLU` on both the full deployment and sub-deployments with individual sites and links removed. This objective assumes that the Service Provider can rebalance traffic quickly using a technology like MPQUIC and so permits different traffic distributions for each failure.

**`Static Traffic Failover`.** This objective is the same as `Dynamic Traffic Failover` but assumes that prefix assignments are fixed during each link and site failure. This objective hence assumes that the Service Provider uses a traffic rebalancing technology like DNS which is too slow to quickly react to failure [1, 87].

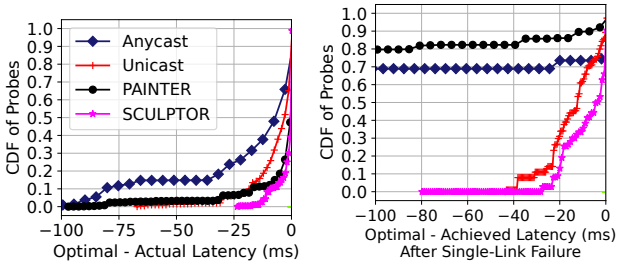
**`Latency Sensitive Services`.** This objective maximizes the amount of traffic within 10ms of its `One-per-Peering` latency, *e.g.,* for latency-sensitive-services that have very tight latency requirements [3]. The objective minimizes the difference with `One-per-Peering` latency instead of absolute latency to isolate `SCULPTOR`'s goal of minimizing routing inefficiency.

**`Traffic Classes`.** This objective routes high-priority traffic with low latency and then as much (latency-insensitive) bulk traffic as possible without congesting high-priority traffic. An additional challenge in the interdomain setting compared to similar intradomain settings [14, 15] is that we cannot use priority queueing to ensure that high-priority traffic is not congested since we do not control queue behavior on the path. We set a limit on the maximum overload on any link so that not all low-priority traffic lands on one link, as this solution would lead to poor low-priority goodput.

**`Traffic Cost Across Sites`.** This objective minimizes the weighted sum of average latency and average site cost, where each site has a unique cost proportional to the traffic volume arriving at the site. It is a simple model of scenarios where Service Providers want to minimize power/carbon/peering infrastructure cost (which can vary by region) or minimize the use of unreliable sites (*e.g.,* dirty power).

## 5.2 On-Internet Experiments

Our Internet deployment (§4.2) experiments demonstrate that `SCULPTOR` achieves latency near `One-per-Peering` (3 ms average improvement) and has zero congestion during link and site failures.




(a) (b)

Figure 5: `SCULPTOR` outperforms other solutions on the Internet deployment, reducing steady-state latency (a) and congestion during link failures (b).

### 5.2.1 `Latency + MLU`

Figure 5a shows a CDF of the difference in latency between each solution and `One-per-Peering` for all RIPE Atlas probes. `SCULPTOR` outperforms other feasible solutions and is only 1.9 ms worse than (the unreasonably expensive) `One-per-Peering` solution on average, closing 65% of the gap left by the next best solution, `PAINTER`, which is 5.5 ms worse.

### 5.2.2 `Dynamic Traffic Failover`

We next demonstrate that `SCULPTOR` robustly handles failures when the Service Provider has a technology for rebalancing traffic across prefixes quickly after the failure (*e.g.,* using MPQUIC). Failures here could represent excessive DDoS traffic on the link/site (thus using the link/site as a sink for the attack), physical failure, resource draining, changes in latency on a path, congestion in the public Internet, and planned maintenance. Hence `SCULPTOR` provides resilience *without reactive BGP changes* which could cause further failure (§2.3)

Figure 5b demonstrates that `SCULPTOR` offers lower latency paths for more probes during single link failure. For the probes whose ingress failed (and not congested), `SCULPTOR` is only 7.9 ms higher latency than (an infeasible) `One-per-Peering` solution and keeps 64.5% of its traffic within 10 ms of it, compared to `unicast`, the only prior approach that does not overload traffic, at 14.2 ms and 38.2%. `PAINTER` and `anycast` struggle, overloading 75.8% and 69.0% of affected traffic.

Site failures yield similar results (not shown): `SCULPTOR` is 13.1 ms higher latency than `One-per-Peering` with 69.0% of affected traffic within 10 ms, while `unicast` is 25.1 ms higher with 39.6%. `PAINTER` again performs poorly, overloading 95.0% of traffic.

`PAINTER`'s performance is much worse in these experiments than in emulations (§5.3) since many probes congested a small number of preferred providers, whereas in emulated routes all providers were equally preferred on average.

## 5.3 Emulated Experiments

We extensively evaluate SCULPTOR against other solutions across all objectives in our emulated deployments. Table 2 demonstrates that SCULPTOR outperforms all other feasible solutions on all deployments. Averages are shown, but SCULPTOR has a 100% win rate by a consistent percentage. Each objective was trained on 10 emulated deployments, each with a random set of users and emulated routing conditions (§4.3). Table 2 reports results for random traffic volumes since they provide more evaluation diversity than APNIC-seeded traffic volumes. Appendix B shows that SCULPTOR also beats on every metric for deployments whose traffic volumes are seeded from APNIC traffic volumes.

### 5.3.1 Latency + MLU

Table 2's Latency + MLU column reports average latency and MLU normalized to anycast. Average latency for SCULPTOR is on average 1.3 ms worse than One-per-Peering, while the next-best solution (PAINTER) is a further 3.3 ms worse than SCULPTOR and has 10% higher latency. SCULPTOR obtains the same maximum link utilization as One-per-Peering.

### 5.3.2 Dynamic Traffic Failover

Table 2's Dynamic Traffic Failover column demonstrates that SCULPTOR shifts traffic without overloading alternate links/sites more effectively than other solutions.

**Performance Under Ingress/Site Failure.** We fail single ingress links and dynamically allocate traffic after the failures as in Section 5.2.2. We find that most solutions have zero congestion since, with 32 sites, there are many backup options that the Service Provider can redirect traffic to. However, users may find this capacity only at a high-latency option: for the users whose ingress failed, SCULPTOR is 6.1 ms worse than One-per-Peering on average, while the next-best solution (PAINTER) is 16.7 ms worse, and unicast is 21.0 ms worse.

Site failures yield more congestion since a site represents up to 5% of global capacity. SCULPTOR congests 5.14% of traffic on average, versus 7.31% for unicast and 11.79% for PAINTER, and its affected users are 1.4 ms worse than One-per-Peering versus 5.7 ms for PAINTER and 5.9 ms for unicast.

**Dynamic Traffic Distributions.** In training for failure robustness, we find that SCULPTOR enables distributing increased load over *existing* infrastructure. We quantify this improved infrastructure utilization under two realistic profiles—flash crowds and diurnal patterns. SCULPTOR *did not explicitly optimize* for these scenarios, demonstrating that SCULPTOR is resilient to changes in underlying traffic distributions.

We identify each client with a single "region" corresponding to the site at which they have the lowest possible latency ingress link. We scale UG traffic volumes until some link overloads in two different ways: (a) in one region at a time (a flash crowd) and (b) on a diurnal peak/trough cycle sampled from a Service Provider's published traffic [17] (shown in Appendix D). In Table 2 we report the critical scale at which overloading occurs relative to anycast's. SCULPTOR absorbs a flash crowd 26% larger than anycast and a diurnal swing 32% larger, against 22% and 26% unicast, the next-best solution.

### 5.3.3 Static Traffic Failover

We fail links and sites but *do not* allow dynamic reallocation—BGP recomputes routes and users route to their next-best preferred option. Hence, SCULPTOR benefits from, but does not require, dynamic traffic rebalancing to retain its benefits.

Under ingress failure SCULPTOR's latency stays within 4.8 ms of One-per-Peering with 0.04% of traffic congested, whereas PAINTER, the next-best solution, is 11.0 ms worse than One-per-Peering with 0.19% congested. Under site failure SCULPTOR is 4.1 ms worse than One-per-Peering and congests 3.94% of traffic, 40% less than the best prior approach (AnyOpt, 6.55%) and 46% less than PAINTER (7.32%), which is 8.5 ms worse than One-per-Peering. unicast and anycast have the same performance since the best solution for unicast is to place all traffic on the anycast prefix, since site failure would blackhole most traffic.

### 5.3.4 Latency Sensitive Services

We also evaluate SCULPTOR's ability to enable latency-sensitive services that have hard latency requirements such as VR/AR with a requirement of 10 ms [3]. Specifically, we try to maximize the amount of traffic to within 10 ms of its lowest latency option. This objective is highly non-linear (a step function) and so is challenging to optimize.

Table 2 shows that SCULPTOR brings 83.7% of traffic within 10 ms of its One-per-Peering latency, 4.7 points more than PAINTER (79.0%), leaving 22% less traffic beyond the threshold, and, compared to anycast, pushes us 83% of the way to the One-per-Peering solution (87.1%).

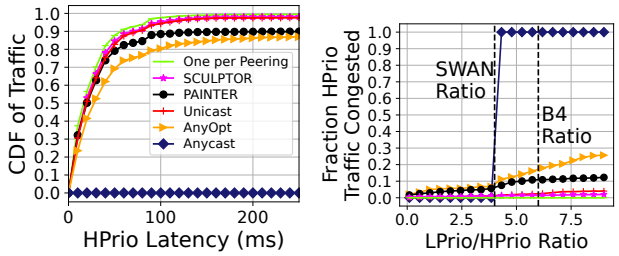
### 5.3.5 Traffic Classes

We also evaluate SCULPTOR's ability to satisfy multiple traffic classes. We split traffic into high and low-priority. The objective is to route high-priority traffic to low-latency routes while limiting congestion on those routes from low-priority traffic. We do not penalize cases where low-priority is congested, but do limit the maximum amount of any traffic on a link to 10× the capacity of the link to avoid solutions where all low-priority is placed on one link, as this solution would lead to low goodput for low-priority traffic (in practice, UGs would lower sending rates in response to congestion). Links are provisioned to 5× the high-priority traffic volume (different from

| **Objectives** | Latency + MLU (§§ 5.2.1 and 5.3.1) | | Dynamic Traffic Failover (§§ 5.2.2 and 5.3.2) | | | Static Traffic Failover (§5.3.3) | | Latency Sensitive Services (§5.3.4) | Traffic Classes (§5.3.5) | | Traffic Cost Across Sites (§5.3.6) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Baseline Solution** | Average Latency (ms) | Maximum link utilization (vs anycast) | Ingress fail: latency (ms) / % cong | Site fail: latency (ms) / % cong | Flash Crowd/Diurnal Intensity (vs anycast) | Ingress fail: latency (ms) / % cong | Site fail: latency (ms) / % cong | % beyond 10ms of optimal | HPrio latency (ms) | Crit bulk ratio | Wgt avg site cost (vs anycast) |
| One-per-Peering [1] | 29.1 | 0.966 | 29.3 / 0.00 | 30.8 / 5.55 | 1.37 / 1.44 | 29.2 / 0.00 | 32.1 / 0.50 | 12.9 | 29.0 | 5.62 | 0.918 |
| SCULPTOR | **30.4** | **0.967** | **30.4 / 0.00** | **32.2 / 5.14** | **1.26 / 1.32** | **34.0 / 0.04** | **36.2 / 3.94** | **16.3** | **31.3** | **4.35** | **0.941** |
| PAINTER | 33.7 | 0.995 | 34.1 / 0.00 | 36.5 / 11.79 | 1.04 / 1.08 | 40.3 / 0.19 | 40.6 / 7.32 | 21.0 | 33.7 | 1.29 | 0.980 |
| unicast | 34.2 | 0.978 | 34.9 / 0.00 | 36.7 / 7.31 | 1.22 / 1.26 | 51.9 / 0.38 | 49.7 / 6.58 | 24.0 | 34.2 | 2.74 | 0.953 |
| AnyOpt | 46.2 | 0.999 | 46.2 / 0.26 | 35.0 / 22.17 | 1.00 / 1.04 | 48.4 / 0.16 | 46.9 / 6.55 | 31.8 | 46.1 | 1.19 | 0.992 |
| anycast | 51.2 | 1.000 | 51.8 / 0.37 | 49.6 / 6.63 | 1.00 / 1.00 | 51.9 / 0.38 | 49.7 / 6.58 | 32.7 | 51.7 | 0.25 | 1.000 |

[1] An unrealistic optimal, included for comparison.

Table 2: SCULPTOR outperforms every feasible solution on every metric. Dark blue (good) and dark red (bad) shading indicate relative performance.




(a) (b)

Figure 6: SCULPTOR routes high-priority traffic with low latency (6a) and minimal congestion from low-priority traffic (6b). We inferred LPrio/HPrio ratios for SWAN [15] and B4 [14] from those papers.

other evaluations), as that is roughly the LPrio/HPrio ratio reported in prior work [14, 15].

Figure 6 zooms in on the dynamics for one representative deployment in Table 2. In Figure 6b we vary the amount of low-priority traffic as a multiple of high-priority traffic volume and compute the fraction of high-priority traffic congested. There is insufficient global capacity to route all traffic for all LPrio/HPrio over 4.0, thus the jump in anycast congestion in Figure 6b. Figure 6a shows a snapshot of HPrio latency for each solution at LPrio/HPrio = 5.0.

Across deployments, SCULPTOR finds strategies that route more low-priority traffic with less congestion than all other approaches. Table 2 demonstrates that SCULPTOR routes 4.35× the high-priority volume in bulk traffic before high-priority traffic congests, 1.6× the next-best solution ,unicast. SCULPTOR simultaneously keeps high-priority latency within 2.3 ms of One-per-Peering (PAINTER: 4.7 ms).

### 5.3.6 Traffic Cost Across Sites

We also evaluate SCULPTOR's ability to jointly minimize both latency and per-site traffic cost, since different sites may cost more or less to serve traffic due to, for example, the availability of power. We solve SCULPTOR using different per-site costs: (a) completely random per-site costs, (b) a model of expensive vs cheap sites where half the sites were 1.2× the cost of other sites, and (c) a model of per-site cost based on the grams of $CO_2$ emitted per kWh of electricity consumed at the site, similar to prior work [98, 99]. We do not claim to have realistic site cost values but use different site-cost choices to demonstrate that SCULPTOR's benefits are not specific to a configuration. Results were similar for each option so we report averages.

SCULPTOR finds advertisements that route traffic to lower-cost sites. On the deployments in Table 2, SCULPTOR's traffic-weighted site cost is 0.941× anycast's, 6% lower and 72% of the way to One-per-Peering (0.918×), versus 0.953× for unicast and 0.980× for PAINTER.

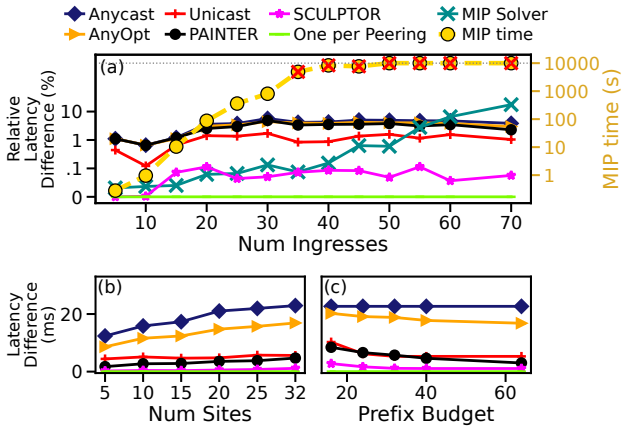



Figure 7: Micro-evaluations. (a) Sub-optimality of Latency + MLU relative to One-per-Peering and MIP-Solver as the deployment grows one ingress at a time. MIP-Solver solve time is the gold dashed line with early-termination latency 10,000 s. Steady-state latency sub-optimality vs. number of sites (b) and prefix budget (c).

## 5.4 Micro-Evaluations

We conduct a set of evaluations/validations to elucidate properties of SCULPTOR including which methodological contributions matter (they all do), a comparison to an "optimal" for small problems, and an investigation of varying parameters.

**Ablation Study.** We solve Dynamic Traffic Failover on 10 emulated 10-site deployments, adding one methodological contribution of SCULPTOR at a time. We normalize PAINTER's performance to 0% and One-per-Peering's to 100%. On

average, our probabilistic objective model §3.4.1 gives 40% of the benefit; searching by descent rather than greedily gives 38% of the benefit; accumulating gradients across iterations adds 13% more, reaching a total of 91%. Adding smart measurements (§3.4.2) still gives us 91% but allows SCULPTOR to reach the same benefit using 15% fewer BGP measurements, saving measurement time.

**Comparison to MIP-Solver.** We use One-per-Peering as a proxy for optimal, but One-per-Peering uses an unrealistic number of prefixes so it is unclear how close to optimal that comparison is. As a closer proxy for an optimal solution, we introduce MIP-Solver which uses a fully determined (emulated) routing function, $R$, and the same prefix budget as SCULPTOR to compute configurations and traffic allocations. Figure 7a demonstrates that this MIP-Solver usually matches SCULPTOR and One-per-Peering to within 0.1%, suggesting that One-per-Peering is essentially the upper bound. Figure 7a also demonstrates that MIP-Solver solve time grows exponentially with size so becomes infeasible for the deployments in Section 5.3 that have thousands of peerings (red x'es show timeouts at 10k seconds).

**Effect of Deployment Size & Prefix Budget.** Here, we vary two key parameters for SCULPTOR to demonstrate that such parameters have a small, intuitive impact on SCULPTOR's relative performance. (Exhaustive evaluation of all parameters is infeasible.) We vary deployment size and prefix budget separately for 10 deployments at each budget/size while optimizing Dynamic Traffic Failover and compare the achieved steady-state latency for different solutions.

Figure 7b and Figure 7c show that SCULPTOR stays within a few ms of One-per-Peering at all deployment sizes and prefix budgets. SCULPTOR's relative benefit compared to PAINTER is the greatest on larger deployments and with smaller prefix budgets which makes sense since those are more challenging problems, so SCULPTOR's better objective prediction and memory can make a larger difference.

## 6 Related Work

**Egress Traffic Engineering.** Prior work noted that traffic from Service Providers to users sometimes experienced suboptimal performance due to BGP's limitations [32, 83, 85]. SCULPTOR works in tandem with these systems, similarly working with BGP, but to engineer ingress traffic. Other work also shifts traffic to other links/sites to lower cost [17, 59, 84]. SCULPTOR adapts this idea in a new way but instead uses the public Internet to carry traffic to lower costs.

**Ingress Traffic Engineering.** We evaluate (§5) and discuss (§2.3) differences to other ingress traffic engineering [1, 6, 13, 46, 48, 49, 82, 100]. FastRoute coarsely balances users across anycast rings to respond to changing conditions [5], giving Service Providers much less control over the specific path users take. PECAN [101] and Tango [102] exhaustively expose paths between endpoints and so may provide resilience to changing conditions if the right paths were exposed. However, exposing all the paths does not scale to our setting since there are too many paths to measure. Recent work balances ingress traffic across sites during failure [87], or optimizes latency by predicting traffic how traffic routes [100, 103], but it is unclear how to extend these methodologies to other objectives.

Other prior work built a BGP playbook to mitigate DDoS attacks [86], but it is unclear if those strategies scale to larger Service Providers (they tested on a few sites/providers). SCULPTOR works for Service Providers with thousands of peering links. Other work/companies create overlay networks and balance load through paths in these overlay networks to satisfy latency requirements [38, 104–107]. SCULPTOR works alongside these overlay networks by advertising the external reachability of these nodes to create better paths through the overlay structure.

**Intradomain Failure Planning.** Service Providers have shown interest in reducing the frequency/impact of failures in their global networks [12, 29, 30, 108]. SCULPTOR works alongside these systems by, for example, enabling Service Providers to shift traffic away from failed components/regions while still retaining good performance. Prior work also shifted traffic during peak times to limit cost/congestion [17], but did so using a private backbone. SCULPTOR's benefits are orthogonal to this prior work and useful for Service Providers without private backbones, as they use the public Internet to realize the same result. Other prior work tried to plan intradomain routes to minimize the impact of k-component failures [17, 109–112]. SCULPTOR solves different challenges that arise due to the lack of visibility/control into potential paths and their capacities in the interdomain setting.

## 7 Conclusions

Unilateral control over intradomain traffic has enabled technology that improves performance, reliability, and flexibility, including fast reroute for quick recovery from failure [113], virtual routing and forwarding for flexible traffic engineering [114], and virtual output queueing for differentiated service [20]. Conversely, interdomain proposals for differentiated services (*e.g.,* intserv/diffserv [115], L4S [116]) and multipath routing (MIRO [117]) have seen little deployment.

SCULPTOR requires no Internet-wide changes. Instead, it leverages black-box modeling and BGP flexibility to unlock a provider's unused global capacity. SCULPTOR is a step towards providing Service Providers with programmable interdomain networking primitives enabling the performance that modern services demand.

## References


[1] Thomas Koch, Shuyue Yu, Ethan Katz-Bassett, Ryan Beckett, and Sharad Agarwal. [n.d.]. PAINTER: Ingress Traffic Engineering and Routing for Enterprise Cloud Networks. In *SIGCOMM 2023*.

[2] Bhaskar Kataria, Palak LNU, Rahul Bothra, Rohan Gandhi, Debopam Bhattacherjee, Venkata N Padmanabhan, Irena Atov, Sriraam Ramakrishnan, Somesh Chaturmohta, Chakri Kotipalli, et al. 2024. Saving Private WAN: Using Internet Paths to Offload WAN Traffic in Conferencing Services. *arXiv preprint arXiv:2407.02037* (2024).

[3] Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Suzan Bayhan, Walter Wong, and Jussi Kangasharju. [n.d.]. Pruning Edge Research With Latency Shears. In *HotNets 2020*.

[4] Jan-Philipp Stauffert, Florian Niebling, and Marc Erich Latoschik. 2018. Effects of latency jitter on simulator sickness in a search task. In *2018 IEEE conference on virtual reality and 3D user interfaces (VR)*.

[5] Ashley Flavel, Pradeepkumar Mani, David Maltz, Nick Holt, Jie Liu, Yingying Chen, and Oleg Surmachev. [n.d.]. FastRoute: A Scalable Load-Aware Anycast Routing Architecture for Modern CDNs. In *NSDI 2015*.

[6] Michael Markovitch, Sharad Agarwal, Rodrigo Fonseca, Ryan Beckett, Chuanji Zhang, Irena Atov, and Somesh Chaturmohta. [n.d.]. TIPSY: Predicting Where Traffic Will Ingress a WAN. In *SIGCOMM 2022*.

[7] Giovane C. M. Moura, Ricardo de Oliveira Schmidt, John Heidemann, Wouter B. de Vries, Moritz Muller, Lan Wei, and Cristian Hesselman. [n.d.]. Anycast vs. DDoS: Evaluating the November 2015 Root DNS Event. In *IMC 2016*.

[8] Matthias Wichtlhuber, Eric Strehle, Daniel Kopp, Lars Prepens, Stefan Stegmueller, Alina Rubina, Christoph Dietzel, and Oliver Hohlfeld. 2022. IXP scrubber: learning from blackholing traffic for ML-driven DDoS detection at scale. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 707–722.

[9] Duncan Riley. 2023. Internet's rapid growth faces challenges amid rising denial-of-service attacks. https://siliconangle.com/2023/09/26/internets-rapid-growth-faces-challenges-amid-rising-ddos-attacks

[10] Chris Stokel-Walker. 2021. Case study: How Akamai weathered a surge in capacity growth. https://increment.com/reliability/akamai-capacity-growth-surge/

[11] Satyajeet Singh Ahuja, Varun Gupta, Vinayak Dangui, Soshant Bali, Abishek Gopalan, Hao Zhong, Petr Lapukhov, Yiting Xia, and Ying Zhang. [n.d.]. Capacity-efficient and uncertainty-resilient backbone network planning with hose. In *SIGCOMM 2021*.

[12] Bingzhe Liu, Colin Scott, Mukarram Tariq, Andrew Ferguson, Phillipa Gill, Richard Alimi, Omid Alipourfard, Deepak Arulkannan, Virginia Jean Beauregard, Patrick Conner, et al. [n.d.]. CAPA: An Architecture For Operating Cluster Networks With High Availability. In *NSDI 2024*.

[13] Xiao Zhang, Tanmoy Sen, Zheyuan Zhang, Tim April, Balakrishnan Chandrasekaran, David Choffnes, Bruce M. Maggs, Haiying Shen, Ramesh K. Sitaraman, and Xiaowei Yang. [n.d.]. AnyOpt: Predicting and Optimizing IP Anycast Performance. In *SIGCOMM 2021*.

[14] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, et al. [n.d.]. B4: Experience with a Globally-Deployed Software Defined WAN. In *SIGCOMM 2013*.

[15] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. [n.d.]. Achieving High Utilization with Software-Driven WAN. In *SIGCOMM 2013*.

[16] Congcong Miao, Zhizhen Zhong, Yunming Xiao, Feng Yang, Senkuo Zhang, Yinan Jiang, Zizhuo Bai, Chaodong Lu, Jingyi Geng, Zekun He, et al. 2024. MegaTE: Extending WAN Traffic Engineering to Millions of Endpoints in Virtualized Cloud. (2024).

[17] David Chou, Tianyin Xu, Kaushik Veeraraghavan, Andrew Newell, Sonia Margulis, Lin Xiao, Pol Mauri Ruiz, Justin Meza, Kiryong Ha, Shruti Padmanabha, et al. [n.d.]. Taiji: Managing Global User Traffic for Large-Scale Internet Services at the Edge. In *SOSP 2019*.

[18] Brandon Schlinker, Todd Arnold, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. PEERING: Virtualizing BGP at the Edge for Research. In *CoNEXT 2019*.

[19] VULTR. 2023. VULTR Cloud. https://vultr.com/

[20] Juniper Networks. 2023. Understanding CoS Virtual Output Queues (VOQs). https://juniper.net/documentation/us/en/software/junos/traffic-mgmt-qfx/topics/concept/cos-qfx-series-voq-understanding.html

[21] Mordor Intelligence. 2022. Network as a Service Market - Growth, Trends, COVID-19 Impact, and Forecasts. https://mordorintelligence.com/industry-reports/network-as-a-service-market-growth-trends-and-forecasts

[22] Krishan Arora. 2023. The Gaming Industry: A Behemoth With Unprecedented Global Reach. https://forbes.com/sites/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach

[23] Cloudflare. 2024. Cloudflare Workers. https://workers.cloudflare.com/

[24] Fastly. 2024. Fastly Compute. https://fastly.com/products/edge-compute

[25] Edgecast. 2020. The CDN Edge brings Compute closer to where it is needed most. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[26] Google Cloud. 2024. Cloud Video Intelligence API. https://cloud.google.com/video-intelligence

[27] Microsoft. 2024. Emirates Global Aluminiium cuts cost of manufacturing AI by 86 percent with the introduction of Azure Stack HCL. https://customers.microsoft.com/en-us/story/1777264680029793974-ega-azure-arc-discrete-manufacturing-en-united-arab-emirates

[28] Yuchen Jin, Sundararajan Renganathan, Ganesh Ananthanarayanan, Junchen Jiang, Venkata N Padmanabhan, Manuel Schroder, Matt Calder, and Arvind Krishnamurthy. [n.d.]. Zooming in on wide-area latencies to a global cloud provider. In *SIGCOMM 2019*.

[29] Ramesh Govindan, Ina Minei, Mahesh Kallahalla, Bikash Koley, and Amin Vahdat. [n.d.]. Evolve or die: High-availability design principles drawn from googles network infrastructure. In *SIGCOMM 2016*.

[30] Umesh Krishnaswamy, Rachee Singh, Nikolaj Bjørner, and Himanshu Raj. [n.d.]. Decentralized cloud wide-area network traffic engineering with BLASTSHIELD. In *NSDI 2022*.

[31] Jeffrey C Mogul, Rebecca Isaacs, and Brent Welch. [n.d.]. Thinking about availability in large service infrastructures. In *HotOS 2017*.

[32] Brandon Schlinker, Hyojeong Kim, Timothy Cui, Ethan Katz-Bassett, Harsha V. Madhyastha, Italo Cunha, James Quinn, Saif Hasan, Petr Lapukhov, and Hongyi Zeng. [n.d.]. Engineering Egress with Edge Fabric: Steering Oceans of Content to the World. In *SIGCOMM 2017*.

[33] Wes Davis. 2023. Netflix ends a three-year legal dispute over Squid Game traffic. https://theverge.com/2023/9/18/23879475/netflix-squid-game-sk-broadband-partnership

[34] Amogh Dhamdhere, David D Clark, Alexander Gamero-Garrido, Matthew Luckie, Ricky KP Mok, Gautam Akiwate, Kabir Gogia, Vaibhav Bajpai, Alex C Snoeren, and Kc Claffy. 2018. Inferring persistent interdomain congestion. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 1–15.

[35] Giovane CM Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. [n.d.]. When the dike breaks: Dissecting DNS defenses during DDoS. In *IMC 2018*.

[36] Matthew Prince. 2013. The DDoS That Knocked Spamhaus Offline (And How We Mitigated It). https://blog.cloudflare.com/the-ddos-that-knocked-spamhaus-offline-and-ho

[37] Sam Kottler. 2018. February 28th DDoS Incident Report. https://github.blog/2018-03-01-ddos-incident-report/

[38] Jinyang Li, Zhenyu Li, Ri Lu, Kai Xiao, Songlin Li, Jufeng Chen, Jingyu Yang, Chunli Zong, Aiyun Chen, Qinghua Wu, et al. [n.d.]. Livenet: a low-latency video transport network for large-scale live streaming. In *SIGCOMM 2022*.

[39] Yarin Perry, Felipe Vieira Frujeri, Chaim Hoch, Srikanth Kandula, Ishai Menache, Michael Schapira, and Aviv Tamar. [n.d.]. DOTE: Rethinking (Predictive) WAN Traffic Engineering. In *NSDI 2023*.

[40] Mark Jackson. 2020. Record Internet Traffic Surge Seen by UK ISPs on Tuesday. https://ispreview.co.uk/index.php/2020/11/record-internet-traffic-surge-seen-by-some-uk-isps-on-tuesday.html

[41] Martyn Landi. 2023. Return of original Fortnite map causes record traffic on Virgin Media O2 network. https://independent.co.uk/tech/fortnite-twitter-b2442476.html

[42] Scott Moritz. 2021. Internet Traffic Surge Triggers Massive Outage on East Coast. https://bloomberg.com/news/articles/2021-01-26/internet-outage-hits-broad-swath-of-eastern-u-s-customers

[43] Doug Madory. 2024. GP Leak Leads to Spike of Misdirected Traffic. https://kentik.com/analysis/BGP-Routing-Leak-Leads-to-Spike-of-Misdirected-Traffic

[44] Andrew Griffin. 2024. Facebook, Instagram, Messenger down: Meta platforms suddenly stop working in huge outage. https://independent.co.uk/tech/facebook-instagram-messenger-down-not-working-latest-b2507376.html

[45] Doug Madory. 2024. Outage Notice From Microsoft. https://twitter.com/DougMadory/status/1768286812894605517

[46] Matt Calder, Ashley Flavel, Ethan Katz-Bassett, Ratul Mahajan, and Jitendra Padhye. [n.d.]. Analyzing the Performance of an Anycast CDN. In *IMC 2015*.

[47] Thomas Koch, Ethan Katz-Bassett, John Heidemann, Matt Calder, Calvin Ardi, and Ke Li. [n.d.]. Anycast in Context: A Tale of Two Systems. In *SIGCOMM 2021*.

[48] Jiangchen Zhu, Kevin Vermeulen, Italo Cunha, Ethan Katz-Bassett, and Matt Calder. [n.d.]. The Best of Both Worlds: High Availability CDN Routing Without Compromising Control. In *IMC 2022*.

[49] Minyuan Zhou, Xiao Zhang, Shuai Hao, Xiaowei Yang, Jiaqi Zheng, Guihai Chen, and Wanchun Dou. [n.d.]. Regional IP Anycast: Deployments, Performance, and Potentials. In *SIGCOMM 2023*.

[50] Patrick Shuff. [n.d.]. Building a Billion User Load Balancer. https://usenix.org/conference/lisa16/conference-program/presentation/shuff

[51] Rupa Krishnan, Harsha V. Madhyastha, Sridhar Srinivasan, Sushant Jain, Arvind Krishnamurthy, Thomas Anderson, and Jie Gao. [n.d.]. Moving Beyond End-to-End Path Information to Optimize CDN Performance. In *IMC 2009*.

[52] Fangfei Chen, Ramesh K. Sitaraman, and Marcelo Torres. [n.d.]. End-User Mapping: Next Generation Request Routing for Content Delivery. In *SIGCOMM 2015*.

[53] Zhihao Li, Dave Levin, Neil Spring, and Bobby Bhattacharjee. [n.d.]. Internet Anycast: Performance, Problems, & Potential. In *SIGCOMM 2018*.

[54] Ann Bednarz. 2023. Global Microsoft cloud-service outage traced to rapid BGP router updates. https://networkworld.com/article/971873/global-microsoft-cloud-service-outage-traced-to-rapid-bgp-router-updates.html

[55] Santosh Janardhan. 2021. More details about the October 4 outage. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[56] Ben Cartwright-Cox. 2023. Grave flaws in BGP Error handling. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[57] Maxime Piraux, Louis Navarre, Nicolas Rybowski, Olivier Bonaventure, and Benoit Donnet. [n.d.]. Revealing the evolution of a cloud provider through its network weather map. In *IMC 2022*.

[58] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. [n.d.]. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *SIGCOMM 2022*.

[59] Rachee Singh, Sharad Agarwal, Matt Calder, and Paramvir Bahl. [n.d.]. Cost-Effective Cloud Edge Traffic Engineering with Cascara. In *NSDI 2021*.

[60] NANOG. 2022. Panel: Buying and Selling IPv4 Addresses. https://youtube.com/watch?v=8FlTJEct9_s

[61] RIPE NCC Staff. 2023. RIS Live. (2023). https://ris-live.ripe.net

[62] Geoff Huston. 2025. How we Measure: IPv6. https://www.potaroo.net/ispcol/2025-11/measure-ipv6.html

[63] 2026. Google IPv6 Statistics. https://www.google.com/intl/en/ipv6/statistics.html

[64] Ticiane Takami. 2021. Project Myriagon: Cloudflare Passes 10,000 Connected Networks. https://blog.cloudflare.com/10000-networks-and-beyond/

[65] Microsoft. 2023. Microsoft Datacenters. https://datacenters.microsoft.com/

[66] Todd Arnold, Jia He, Weifan Jiang, Matt Calder, Italo Cunha, Vasileios Giotsas, and Ethan Katz-Bassett. [n.d.]. Cloud Provider Connectivity in the Flat Internet. In *IMC 2020*.

[67] Costin Raiciu, Christoph Paasch, Sebastien Barre, Alan Ford, Michio Honda, Fabien Duchene, Olivier Bonaventure, and Mark Handley. [n.d.]. How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP. In *NSDI 2012*.

[68] Quentin De Coninck and Olivier Bonaventure. [n.d.]. Multipath QUIC: Design and Evaluation. In *CoNEXT 2017*.

[69] Quentin De Coninck and Olivier Bonaventure. 2021. Multiflow QUIC: A generic multipath transport protocol. *IEEE Communications Magazine* (2021).

[70] Patrick Sattler, Juliane Aulbach, Johannes Zirngibl, and Georg Carle. [n.d.]. Towards a Tectonic Traffic Shift? Investigating Apple's New Relay Network. In *IMC 2022*.

[71] Matt Calder, Ryan Gao, Manuel Schröder, Ryan Stewart, Jitendra Padhye, Ratul Mahajan, Ganesh Ananthanarayanan, and Ethan Katz-Bassett. [n.d.]. Odin: Microsoft's Scalable Fault-Tolerant CDN Measurement System. In *NSDI 2018*.

[72] Wouter B. De Vries, Ricardo de O. Schmidt, Wes Hardaker, John Heidemann, Pieter-Tjerk de Boer, and Aiko Pras. [n.d.]. Broad and Load-Aware Anycast Mapping with Verfploeter. In *IMC 2017*.

[73] Palak Goenka, Kyriakos Zarifis, Arpit Gupta, and Matt Calder. 2022. Towards client-side active measurements without application control. *SIGCOMM CCR 2022* (2022).

[74] Pavlos Sermpezis and Vasileios Kotronis. 2019. Inferring catchment in internet routing. (2019).

[75] [n.d.]. Shannon Entropy. https://en.wikipedia.org/wiki/Entropy_(information_theory)

[76] Sophie Langer. 2021. Approximating smooth functions by deep neural networks with sigmoid activation function. *Journal of Multivariate Analysis 2021* (2021).

[77] Yoav Shechtman, Amir Beck, and Yonina C Eldar. 2014. GESPAR: Efficient phase retrieval of sparse signals. *IEEE transactions on signal processing* (2014).

[78] Yurii Nesterov et al. 2018. *Lectures on convex optimization*. Vol. 137. Springer.

[79] Haochuan Li, Jian Qian, Yi Tian, Alexander Rakhlin, and Ali Jadbabaie. 2024. Convex and non-convex optimization under generalized smoothness. *Advances in Neural Information Processing Systems* 36 (2024).

[80] SciPy developers. 2024. SciPy linprog (HiGHS). https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html

[81] RIPE NCC Staff. 2015. RIPE Atlas: A Global Internet Measurement Network. *Internet Protocol Journal* (2015).

[82] Hongqiang Harry Liu, Raajay Viswanathan, Matt Calder, Aditya Akella, Ratul Mahajan, Jitendra Padhye, and Ming Zhang. [n.d.]. Efficiently Delivering Online Services over Integrated Infrastructure. In *NSDI 2016*.

[83] Kok-Kiong Yap, Murtaza Motiwala, Jeremy Rahe, Steve Padgett, Matthew Holliman, Gary Baldus, Marcus Hines, Taeeun Kim, Ashok Narayanan, Ankur Jain, et al. [n.d.]. Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering. In *SIGCOMM 2017*.

[84] Zheng Zhang, Ming Zhang, Albert G. Greenberg, Y. Charlie Hu, Ratul Mahajan, and Blaine Christian. [n.d.]. Optimizing Cost and Performance in Online Service Provider Networks. In *NSDI 2010*.

[85] Raul Landa, Lorenzo Saino, Lennert Buytenhek, and João Taveira Araújo. [n.d.]. Staying Alive: Connection Path Reselection at the Edge. In *NSDI 2021*.

[86] ASM Rizvi, Leandro Bertholdo, João Ceron, and John Heidemann. [n.d.]. Anycast Agility: Network Playbooks to Fight DDoS. In *USENIX Security Symposium 2022*.

[87] Jiangchen Zhu, Tom Koch, Ilgar Mammadov, Shuyue Yu, Kevin Vermeulen, Matt Calder, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. The New (Pareto) Frontier of Cloud Routing: High Availability, Precise Control, or Configuration Stability-Choose Two. *CoNEXT 2025*.

[88] Nikolaos Chatzis, Georgios Smaragdakis, Anja Feldmann, and Walter Willinger. 2013. There is more to IXPs than meets the eye. *SIGCOMM Computer Communication Review* (2013).

[89] Geoff Huston. 2014. How Big is that Network? labs.apnic.net/?p=526

[90] Loqman Salamatian, Calvin Ardi, Vasileios Giotsas, Matt Calder, Ethan Katz-Bassett, and Todd Arnold. [n.d.]. What's in the Dataset? Unboxing the APNIC per AS User Population Dataset. In *IMC 2024*.

[91] Tobias Flach, Nandita Dukkipati, Andreas Terzis, Barath Raghavan, Neal Cardwell, Yuchung Cheng, Ankur Jain, Shuai Hao, Ethan Katz-Bassett, and Ramesh Govindan. [n.d.]. Reducing web latency: the virtue of gentle aggression. In *SIGCOMM 2013*.

[92] David Wetherall, Abdul Kabbani, Van Jacobson, Jim Winget, Yuchung Cheng, Charles B Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, and Amin Vahdat. [n.d.]. Improving Network Availability with Protective ReRoute. In *SIGCOMM 2023*.

[93] Danilo Giordano, Danilo Cicalese, Alessandro Finamore, Marco Mellia, Maurizio Munafò, Diana Zeaiter Joumblatt, and Dario Rossi. [n.d.]. A First Characterization of Anycast Traffic from Passive Traces. In *TMA 2016*.

[94] Danilo Cicalese, Jordan Augé, Diana Joumblatt, Timur Friedman, and Dario Rossi. [n.d.]. Characterizing IPv4 Anycast Adoption and Deployment. In *CoNEXT 2015*.

[95] Verizon. 2020. Edgecast. https://verizondigitalmedia.com/media-platform/delivery/network/

[96] Matthew Prince. 2013. Load Balancing without Load Balancers. https://blog.cloudflare.com/cloudflares-architecture-eliminating-single-p/

[97] Jing'an Xue, Weizhen Dang, Haibo Wang, Jilong Wang, and Hui Wang. [n.d.]. Evaluating Performance and Inefficient Routing of an Anycast CDN. In *International Symposium on Quality of Service 2019*.

[98] Electicity Maps. 2026. Electicity Maps. https://www.electricitymaps.com/data

[99] Sawsan El-Zahr, William Nathan, and Noa Zilberman. [n.d.]. Carbon-Intelligent Content Scheduling in CDNs. In *ANRW 2025*.

[100] Minyuan Zhou, Yuning Chen, Jiaqi Zheng, Yifei Xu, Pan Hu, Yongping Tang, Wendong Yin, Jie Lin, Qingyan Yu, Yuanchao Su, Guihai Chen, Wanchun Dou, Songwu Lu, and Wan Du. [n.d.]. AnyPro: Preference-Preserving Anycast Optimization based on Strategic AS-Path Prepending. In *NSDI 2026*.

[101] Vytautas Valancius, Bharath Ravi, Nick Feamster, and Alex C Snoeren. [n.d.]. Quantifying the Benefits of Joint Content and Network Routing. In *SIGMETRICS 2013*.

[102] Henry Birge-Lee, Sophia Yoo, Benjamin Herber, Jennifer Rexford, and Maria Apostolaki. [n.d.]. {TANGO}: Secure Collaborative Route Control across the Public Internet. In *NSDI 2024*.

[103] Minyuan Zhou, Jiaqi Zheng, Congying Wang, Guihai Chen, and Wanchun Dou. [n.d.]. AnyTuner: Optimizing IP Anycast Performance via Strategic BGP Routing Policy. In *ICNP 2025*.

[104] Subspace. 2022. Optimize Your Network on Subspace. https://subspace.com/solutions/reduce-internet-latency

[105] INAP. 2022. INAP Network Connectivity. https://inap.com/network/

[106] Akamai. 2022. Akamai Secure Access Service Edge. https://akamai.com/resources/akamai-secure-access-service-edge-sase

[107] Cloudflare. 2022. Argo Smart Routing. https://cloudflare.com/products/argo-smart-routing/

[108] John P Eason, Xueqi He, Richard Cziva, Max Noormohammadpour, Srivatsan Balasubramanian, Satyajeet Singh Ahuja, and Biao Lu. [n.d.]. Hose-based cross-layer backbone network design with Benders decomposition. In *SIGCOMM 2023*.

[109] Ye Wang, Hao Wang, Ajay Mahimkar, Richard Alimi, Yin Zhang, Lili Qiu, and Yang Richard Yang. [n.d.]. R3: resilient routing reconfiguration. In *SIGCOMM 2010*.

[110] Hongqiang Harry Liu, Srikanth Kandula, Ratul Mahajan, Ming Zhang, and David Gelernter. [n.d.]. Traffic engineering with forward fault correction. In *SIGCOMM 2014*.

[111] Yiyang Chang, Chuan Jiang, Ashish Chandra, Sanjay Rao, and Mohit Tawarmalani. 2019. Lancet: Better network resilience by designing for pruned failure sets. (2019).

[112] Chuan Jiang, Sanjay Rao, and Mohit Tawarmalani. [n.d.]. PCF: provably resilient flexible routing. In *SIGCOMM 2020*.

[113] IETF. 2005. Fast Reroute Extensions to RSVP-TE for LSP Tunnels. https://datatracker.ietf.org/doc/html/rfc4090

[114] IETF. 2006. BGP/MPLS IP Virtual Private Networks (VPNs). https://datatracker.ietf.org/doc/html/rfc4364

[115] IETF. 2000. A Framework for Integrated Services Operation over Diffserv Networks. https://datatracker.ietf.org/doc/html/rfc2998

[116] IETF. 2023. Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture. https://datatracker.ietf.org/doc/rfc9330/

[117] Wen Xu and Jennifer Rexford. [n.d.]. MIRO: Multi-Path Interdomain Routing. In *SIGCOMM 2006*.

[118] Sharad Agarwal, Chen-Nee Chuah, Supratik Bhattacharyya, and Christophe Diot. [n.d.]. Impact of BGP Dynamics on Router CPU Utilization. In *PAM 2004*.

| | |
|---|---|
| *Shared by all objectives* | |
| $V$ | Total demand, $\sum_{\texttt{UG}} v(\texttt{UG})$ |
| $f_l(R,w)$ | Load on link $l$ |
| $\Lambda(R,w)$ | Traffic-weighted mean latency |
| $G_0$ | The latency-and-MLU objective (Eq. (3)) |
| *Latency and MLU (Eq. (3))* | |
| $M$ | Maximum link utilization |
| $\beta$ | Weight on utilization vs. propagation delay |
| *Failover (Eqs. (4) and (5))* | |
| $F_l$ | Failure mask zeroing advertisements on $l$ |
| $A_l$ | $A * F_l$: configuration with $l$ withdrawn |
| $\alpha_l$ | Weight on the failure of $l$, *e.g.,* its likelihood |
| $w^{(l)}$ | Post-failure allocation (dynamic failover only) |
| *Latency-sensitive services (Eq. (6))* | |
| $\mathcal{L}^*(\texttt{UG})$ | UG's lowest-latency reachable ingress |
| $\tau$ | Latency budget above $\mathcal{L}^*$ (10 ms) |
| *Multiple traffic classes (Eq. (7))* | |
| $w_H, w_B$ | High-priority and bulk allocations |
| $V_H, f_l^H$ | High-priority demand and per-link load |
| $\gamma$ | Weight on high-priority congestion vs. latency |
| $\kappa$ | Cap on per-link overload |
| *Per-site traffic cost (Eq. (8))* | |
| $site(l)$ | Site terminating link $l$ |
| $\rho_s$ | Unit traffic cost at site $s$ |
| $\delta$ | Weight on site cost vs. latency |

Table 3: Symbols used in the objective functions. Core notation appears in Table 1.

# A Methodology Clarifications

## A.1 Definition of Terms

Table 3 lists terms and expressions used in Section 3 and Section A.2, in the order they appear.

## A.2 Objective Functions

Every objective we evaluate is an instance of Equation (2) and differs only in $G$. Throughout, $V = \sum_{\texttt{UG}} v(\texttt{UG})$ is total demand, the load on link $l$ is $f_l(R,w) = \sum_{R(A)(p,\texttt{UG})=l} w(p,\texttt{UG})$, its utilization is $f_l/c(l)$, and traffic-weighted mean latency is $\Lambda(R,w) = \frac{1}{V}\sum_{p,\texttt{UG}} \mathcal{L}(\texttt{UG}, R(A)(p,\texttt{UG}))\, w(p,\texttt{UG})$. All are subject to the constraints of Equation (1).

**Latency and MLU.**

$$G = \Lambda(R,w) + \beta M \quad \text{s.t.} \quad \frac{f_l(R,w)}{c(l)} \le M \;\; \forall l \tag{3}$$

Minimizing forces $M$ to the maximum link utilization; $\beta$ sets the operator's tradeoff between propagation delay and headroom.

**Dynamic traffic failover.** Let $A_l = A * F_l$ denote the configuration with link or site $l$ withdrawn, and let $G_0$ be Equation (3). Each failure scenario gets its *own* allocation $w^{(l)}$, modelling an operator who can rebalance traffic quickly (*e.g.,* over MPQUIC):

$$\min_{w} G_0(R(A),w) \;+\; \sum_{l} \alpha_l \min_{w^{(l)}} G_0(R(A_l), w^{(l)}) \tag{4}$$

**Static traffic failover.** The same term, with a *single* allocation shared across the nominal case and every failure, modelling an operator restricted to slow rebalancing (*e.g.,* DNS):

$$\min_{w} \Big[ G_0(R(A),w) \;+\; \sum_{l} \alpha_l G_0(R(A_l),w) \Big] \tag{5}$$

Equation (5)'s feasible set is a restriction of Equation (4)'s, so for any $A$ the static objective is never smaller than the dynamic one—an invariant we assert when generating results. We use $\alpha_l = 0.1$.

**Latency-sensitive services.** Let $\mathcal{L}^*(\texttt{UG}) = \min_l \mathcal{L}(\texttt{UG}, l)$ over ingresses reachable by UG, and $\tau = 10$ ms. We maximize traffic served within $\tau$ of that best option:

$$G = -\frac{1}{V}\sum_{p,\texttt{UG}} w(p,\texttt{UG}) \cdot \mathbf{1}[\,\mathcal{L}(\texttt{UG}, R(A)(p,\texttt{UG})) - \mathcal{L}^*(\texttt{UG}) \le \tau\,] \tag{6}$$

The indicator makes this a step function, and hence a harder search than the others.

**Multiple traffic classes.** Split demand into high-priority and bulk, with allocations $w_H$, $w_B$ and loads $f_l^H, f_l$. With $\gamma$ weighting congestion against latency and $\kappa$ capping per-link overload:

$$G = \Lambda_H(R,w_H) + \gamma \frac{1}{V_H} \sum_{l: f_l > c(l)} f_l^H \quad \text{s.t.} \quad \frac{f_l}{c(l)} \le \kappa \;\; \forall l \tag{7}$$

The cap prevents degenerate solutions that pile all bulk traffic onto one link.

**Per-site traffic cost.** Site $s$ serves traffic at unit cost $\rho_s$, and $site(l)$ is the site terminating link $l$:

$$G = \Lambda(R,w) + \delta \frac{1}{V} \sum_{l} \rho_{site(l)} f_l(R,w) \tag{8}$$

## A.3 One Iteration

Putting the pieces together, one iteration of SCULPTOR proceeds as follows (Figure 3).

1. Threshold the relaxed configuration $\tilde{A}$ at 0.5 to obtain the current advertisement $mathitA$ (§3.5.2).
2. If the model's entropy is high, or enough iterations have passed since the last measurement, advertise $A$ via BGP and observe the ingress each UG uses for each prefix compared to what was available to update the routing model (§3.4.2).
3. Draw $K$ routing functions $R$ from the current model, one ingress per UG and prefix (§3.4.1).

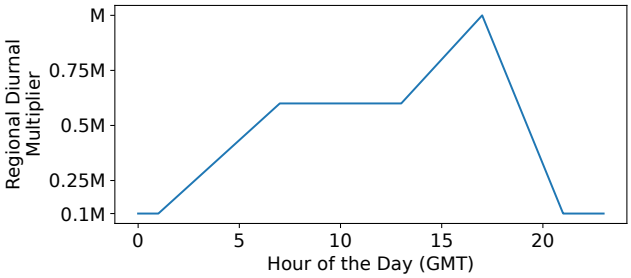



Figure 8: The diurnal traffic shape we apply in Section 5.3.2, taken from prior work [17].

4. For a random subset of entries $(p,l)$ of $\tilde{A}$, solve the traffic allocation with $A(p,l)$ set to 1 and to 0 under each sampled $R$ (§3.6); the averaged differences, passed through a sigmoid, are the gradient entries.
5. Take a Nesterov step on $\tilde{A}$.
6. Repeat until the measurement budget is exhausted or the objective stops improving (§3.5.3).

## B Emulation Results With APNIC Volumes

Table 4 shows the same emulated evaluation results as Table 2, except using APNIC population estimates as traffic volumes [89]. SCULPTOR still outperforms every other baseline on every metric.

## C Considerations for Real-World Deployments

In Section 4 we discuss how we prototype SCULPTOR. Here we give a high-level overview of how a real cloud can practically, safely implement SCULPTOR. The measurement, advertisement, and learning process described in Section 3 is meant to be carried out before any production traffic is shifted to the prefixes SCULPTOR is configuring, as in prior work [1, 13, 46].

SCULPTOR requires latency measurements to each ingress. To obtain these, the Service Provider should direct clients to measure latency towards a test prefix using a measurement system similar to those found in the literature [52, 71–73]. Note that this process *does not involve production traffic*. The Service Provider should then advertise the test prefix to one ingress at a time and record client latencies to the ingress.

With these initial measurements, Service Providers can initialize the learning process described in Section 3.4.2 using test prefixes. Prior work also shows that BGP routing changes minimally disrupt routers [118], so SCULPTOR's learning process using test prefixes does not affect production traffic.

After convergence, SCULPTOR can slowly shift production traffic over to the optimal learned configuration. Note that, since most traffic is already routed along a good path [1, 13, 46, 51], SCULPTOR may only need to shift a small fraction of traffic or do so infrequently [6], mitigating the blast radius of configuration changes. The learning process is very infrequent, as prior work found that advertisements do not need to be updated even after a month [1].

## D Diurnal Traffic Shape

In Figure 8, we show the shape of our diurnal curve taken from observed diurnal traffic patterns at a Service Provider [17]. We use this diurnal traffic pattern to evaluate how well balances traffic across sites (§5.3.2). Specifically, we linearly interpolate 6 points on the purple curve (Edge Node 1) in Figure 1 of that paper, capturing the essential peaks and troughs.

| **Objectives** | Latency + MLU (§§ 5.2.1 and 5.3.1) | | Dynamic Traffic Failover (§§ 5.2.2 and 5.3.2) | | | Static Traffic Failover (§5.3.3) | | Latency Sensitive Services (§5.3.4) | Traffic Classes (§5.3.5) | | Traffic Cost Across Sites (§5.3.6) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Baseline Solution** | Average Latency (ms) | Maximum link utilization (vs anycast) | Ingress fail: latency (ms) / % cong | Site fail: latency (ms) / % cong | Flash Crowd/Diurnal Intensity (vs anycast) | Ingress fail: latency (ms) / % cong | Site fail: latency (ms) / % cong | % beyond 10ms of optimal | HPrio latency (ms) | Crit bulk ratio | Wgt avg site cost (vs anycast) |
| One-per-Peering [1] | 30.1 | 0.990 | 30.2 / 0.00 | 32.1 / 6.90 | 1.10 / 1.11 | 30.3 / 0.00 | 33.4 / 0.36 | 11.1 | 30.1 | 16.00 | 0.904 |
| SCULPTOR | **33.0** | **0.991** | **32.2 / 0.00** | **33.2** / 8.08 | **1.06 / 1.09** | **37.6 / 0.11** | **39.2 / 4.00** | **15.9** | **32.2** | **16.00** | **0.937** |
| PAINTER | 34.8 | 1.000 | 35.3 / 3.19 | 38.0 / 25.34 | 1.00 / 1.00 | 45.5 / 0.44 | 44.4 / 5.88 | 18.2 | 34.8 | 3.33 | 0.969 |
| unicast | 33.7 | 0.992 | 33.9 / 0.00 | 34.5 / 7.60 | 1.06 / 1.09 | 47.7 / 0.85 | 44.8 / 6.71 | 17.9 | 33.7 | 16.00 | 0.949 |
| AnyOpt | 43.4 | 1.000 | 43.0 / 1.26 | 35.7 / 14.90 | 1.00 / 1.00 | 45.6 / 0.43 | 43.5 / 6.32 | 25.0 | 43.4 | 4.25 | 0.991 |
| anycast | 48.5 | 1.000 | 47.7 / 0.83 | 45.0 / **6.62** | 1.00 / 1.00 | 47.7 / 0.85 | 44.8 / 6.71 | 26.0 | 48.5 | 4.00 | 1.000 |

[1] An unrealistic optimal, included for comparison.

Table 4: Performance of all methods across objectives on the APNIC-volume deployments. Dark blue (good) and dark red (bad) shading indicate relative performance.